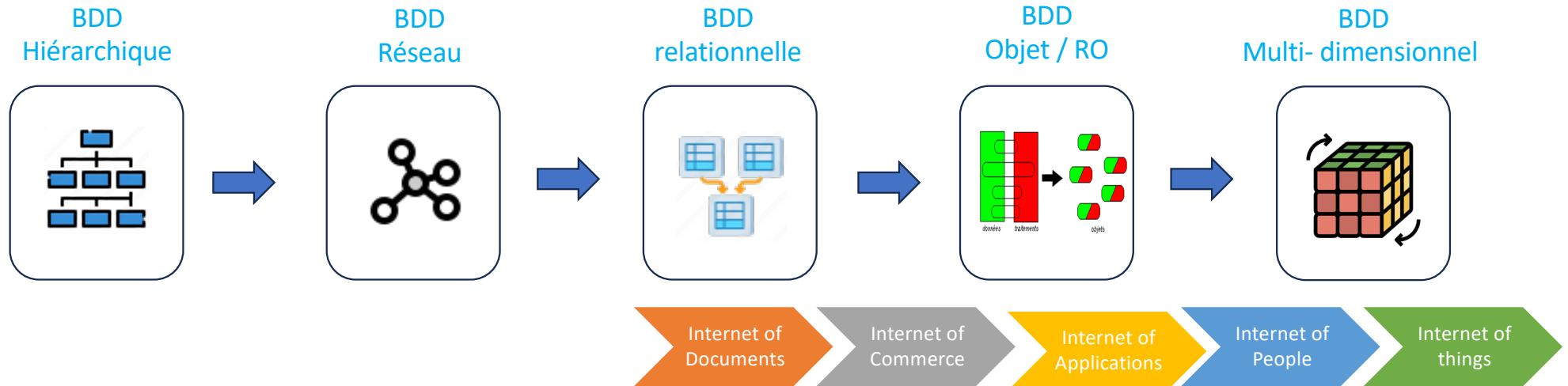


Evolution des architectures des bases de données



L'évolution des bases de données



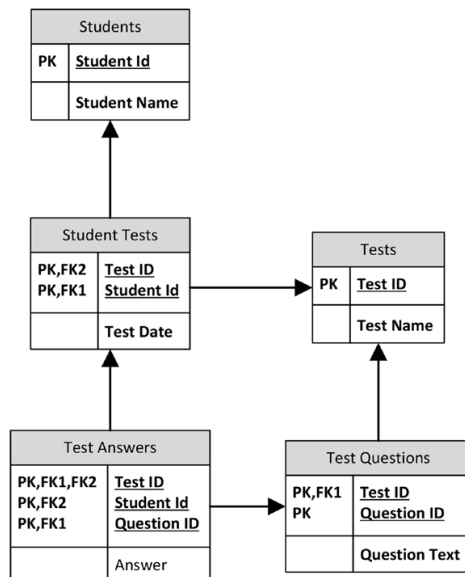
À mesure que les bases de données grandissent en taille, il devient nécessaire d'investir dans du matériel haute performance pour maintenir des performances optimales.
Ce processus est connu sous le nom d'**extensibilité verticale** (scale-up).

Rappel sur le modèle relationnel

Un-normalized data

Test scores
Student Name
Test Name
Test Date
Answer 1
Answer 2
Answer 3
Answer 4
Answer 5
Answer 6
Answer N

Normalized data



- ❑ Le modèle relationnel fait son apparition en 1970, suite à la publication du Dr Edgar F. Codd, chercheur chez IBM.
- ❑ Il repose sur une base mathématique solide, fondée sur l'algèbre relationnelle et la théorie des ensembles.
- ❑ Les données sont représentées sous forme de relations (ou tables), organisées en lignes (tuples) et colonnes (attributs).
- ❑ Le modèle introduit la normalisation, un principe visant à réduire les redondances et prévenir les incohérences, en structurant les données selon des formes normales.

Rappel sur le modèle relationnel

- ❑ Le modèle relationnel spécifie comment organiser les données sous forme de tables, mais il ne traite pas la gestion des accès concurrents ni la cohérence en environnement multi-utilisateur.
- ❑ Pour résoudre ce problème, Jim Gray (chercheur reconnu dans le domaine des bases de données) propose le modèle transactionnel.
- ❑ Une transaction, c'est un ensemble d'opérations (par exemple : insérer un client, mettre à jour un stock, valider une commande) que l'on veut réaliser de manière sûre et cohérente.

```
1  START TRANSACTION;  
2  •  update account Set Balance = Balance + 100 where AccountId = 1;  
3  •  update account Set Balance = Balance - 100 where AccountId = 5;  
4  •  COMMIT;
```

- ❑ Chaque transaction doit respecter 4 propriétés essentielles, résumées par l'acronyme ACID

Rappel sur le modèle relationnel

A – Atomicité :

La transaction est tout ou rien : si une opération échoue, la transaction est annulée. Aucune modification partielle ne doit rester.

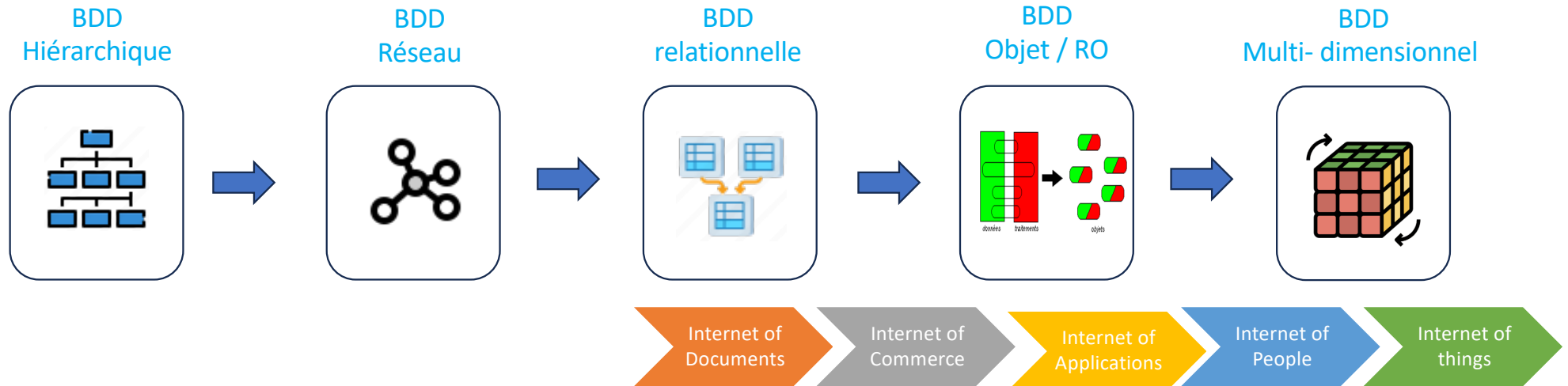


I – Isolation : Plusieurs transactions peuvent s'exécuter en même temps sans se gêner. Elles doivent se comporter comme si elles étaient traitées l'une après l'autre.

C – Cohérence : Les règles de gestion (contraintes, intégrité...) sont toujours respectées : la base passe d'un état valide/cohérent à un autre état cohérent.

D – Durabilité : Une fois validée (COMMIT), une transaction est définitive : ses modifications ne disparaîtront pas, même en cas de panne.

L'évolution des bases de données



À mesure que les bases de données grandissent en taille, il devient nécessaire d'investir dans du matériel haute performance pour maintenir des performances optimales.
Ce processus est connu sous le nom d'**extensibilité verticale** (scale-up).

Les applications 3.0



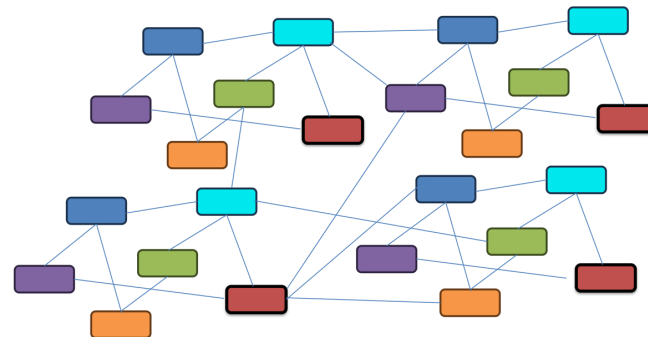
Conteneur



Audit et logs



Orchestration des conteneurs



Performance / Supervision



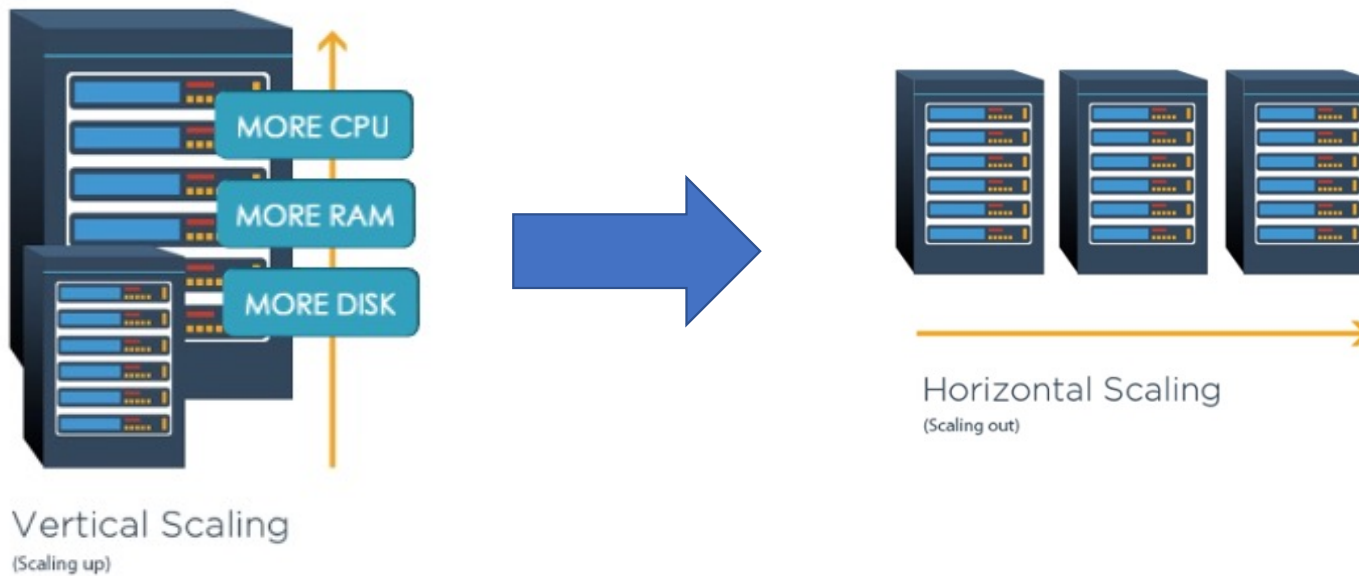
GitLab

Automatisation des processus de compilation



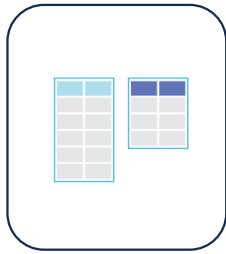
Bus de communication

Les bases de données 3.0



Les bases de données 3.0

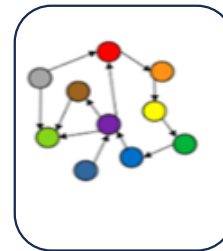
Les bases de données Not Only SQL (NoSQL)
On privilégie la performance et l'extensibilité horizontale.



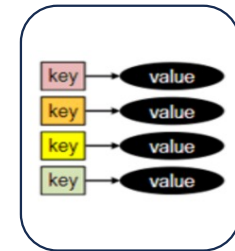
BDD Orientée Colonne



BDD Orientée Document



BDD Orientée Graph

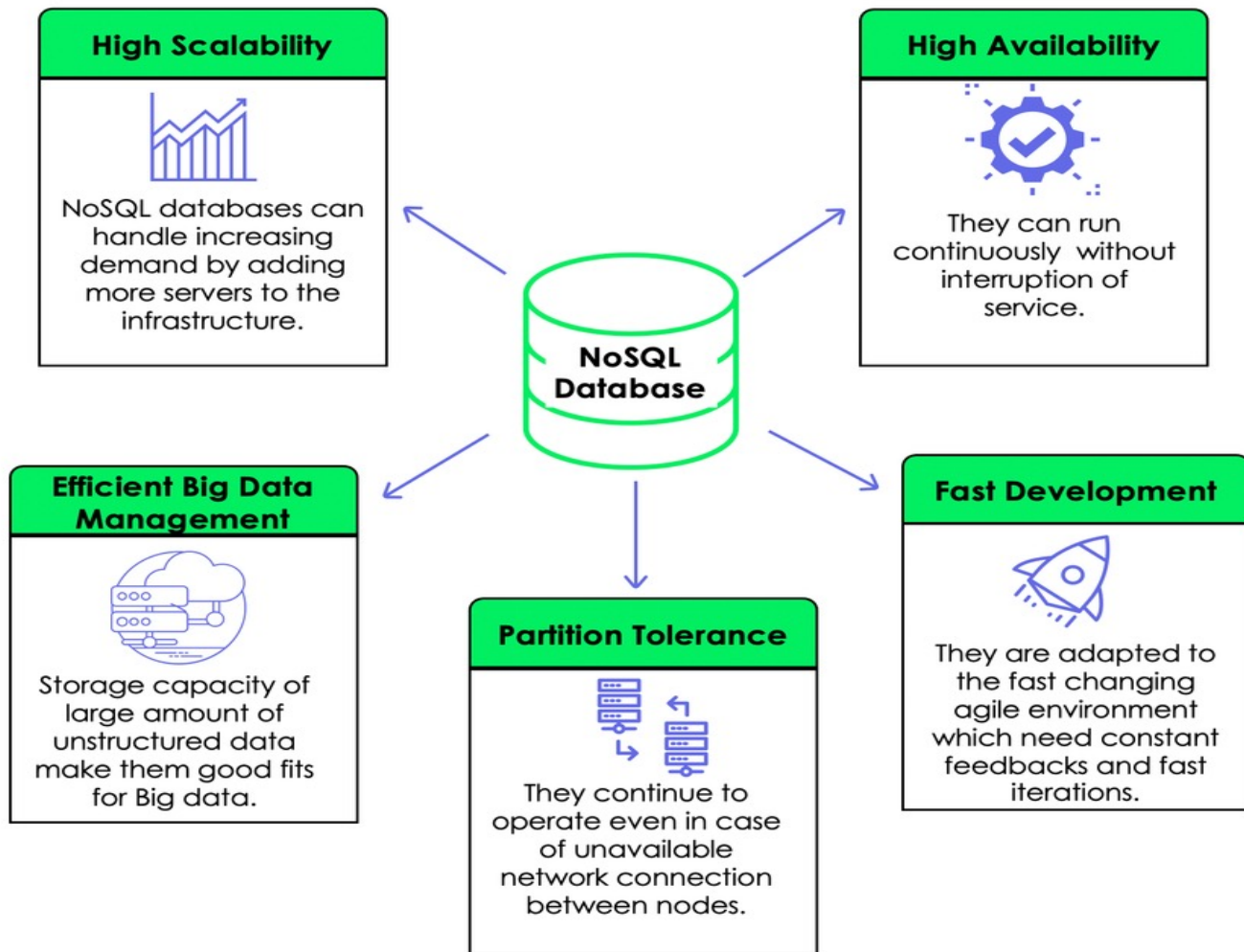


BDD Orientée Clé-valeur

***What
the
.....?***

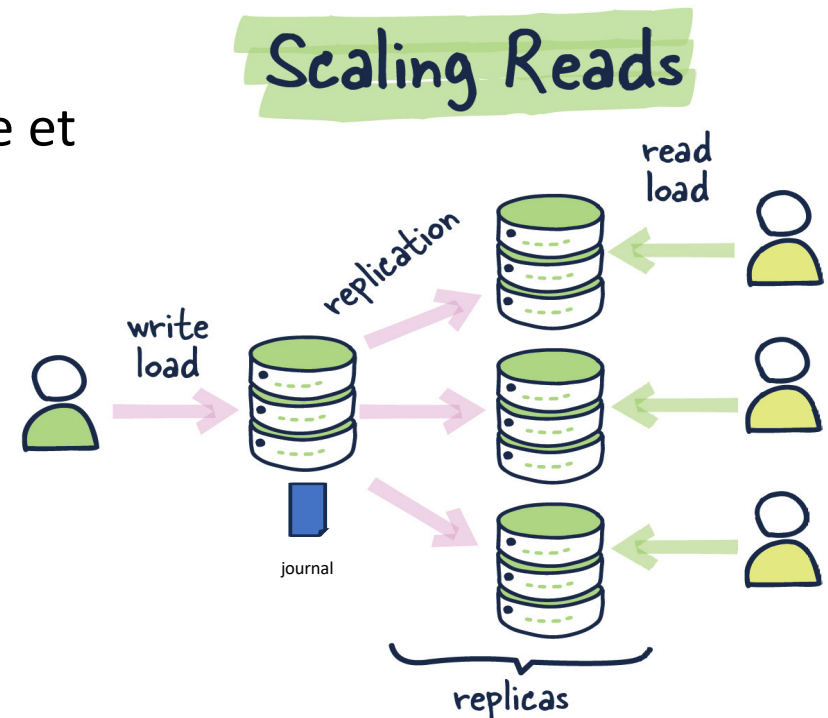


Le NoSQL

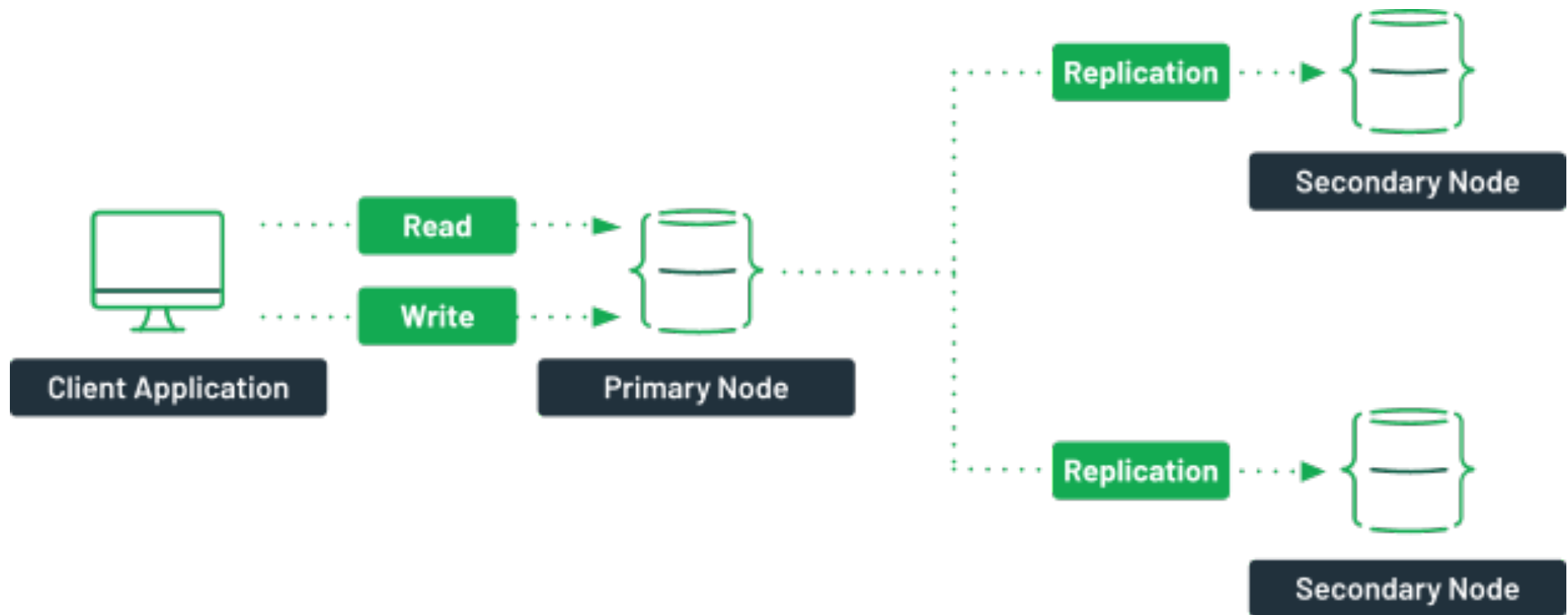


Réplication : Disponibilité & Répartition de charge

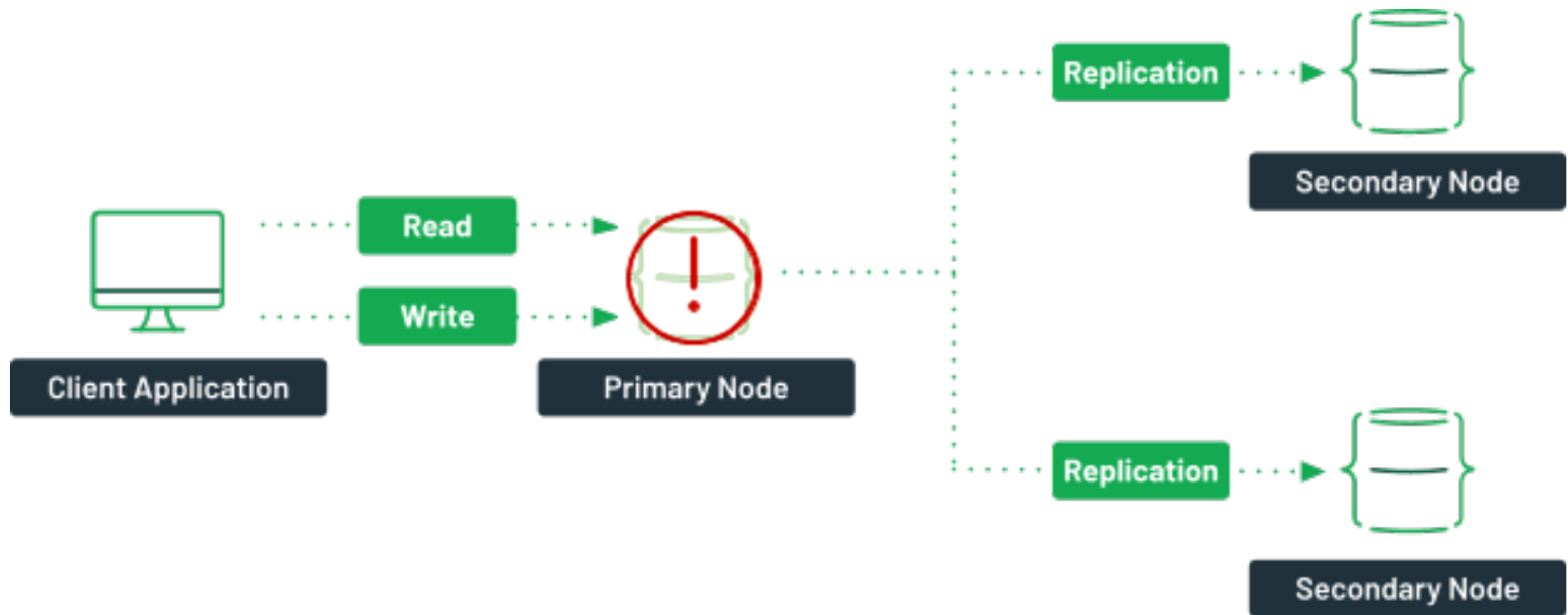
- ❑ Objectif principal : répartir la charge en lecture et améliorer la disponibilité.
- ❑ Modèle courant à l'ère du Web 2.0.
- ❑ Les données sont répliquées via des journaux.



Réplication : Disponibilité & Répartition de charge



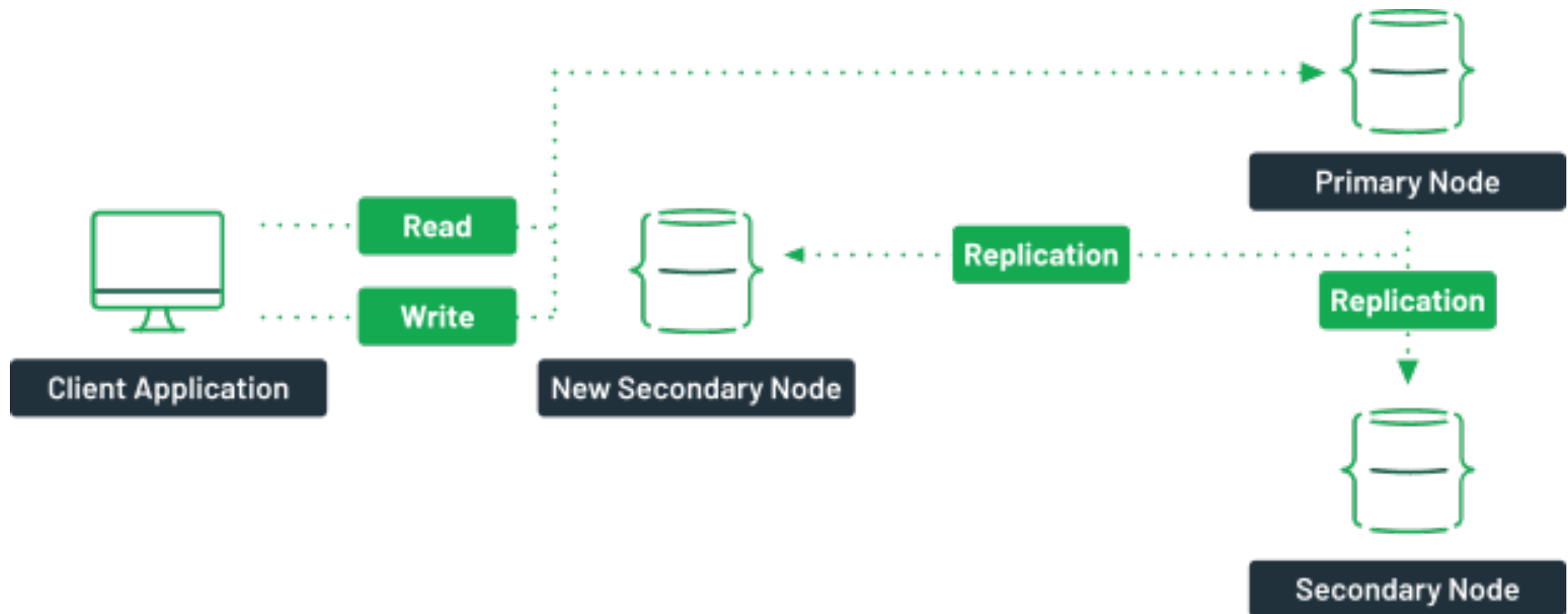
Réplication : Disponibilité & Répartition de charge



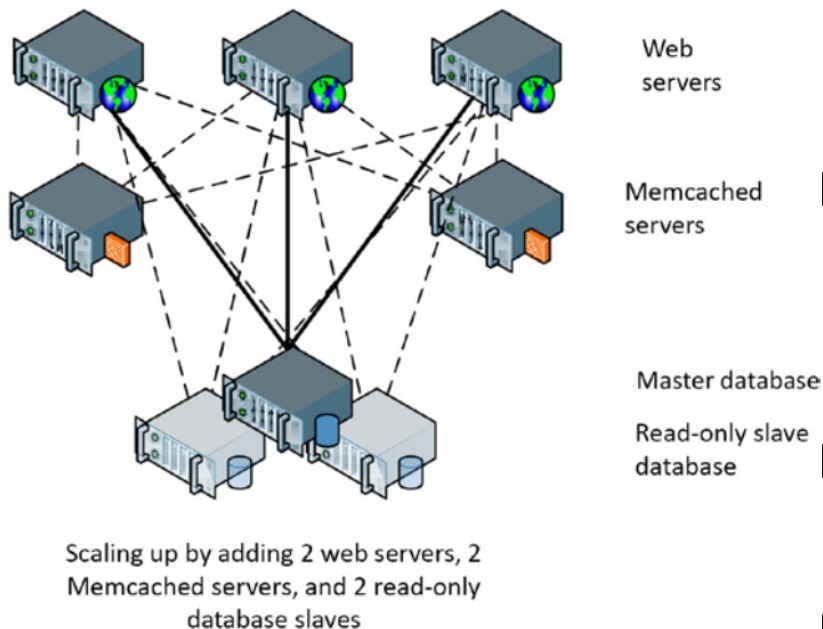
Réplication : Disponibilité & Répartition de charge



Réplication : Disponibilité & Répartition de charge



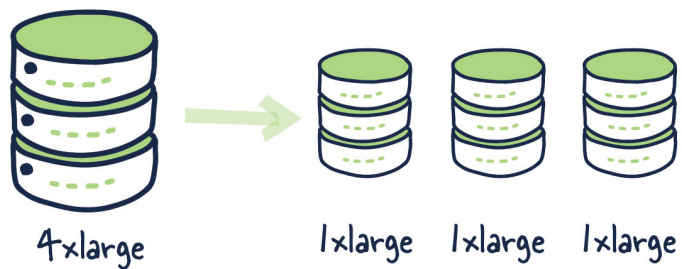
Réplication : Disponibilité & Répartition de charge



- ☐ Lorsqu'une donnée n'est pas présente dans le cache Memcached, l'application la récupère en consultant les réplicas de la base de données, puis elle l'enregistre dans Memcached pour les futures requêtes.
- ☐ Si la donnée est déjà en cache, elle est fournie instantanément par Memcached, un store clé-valeur ultra-rapide, permettant un accès en quelques nanosecondes.
- ☐ En combinant Memcached et la réplication, on peut absorber une charge de lecture très élevée,
- ☐ grâce à la répartition sur plusieurs serveurs et à la rapidité du cache.

Sharding

Horizontal scaling



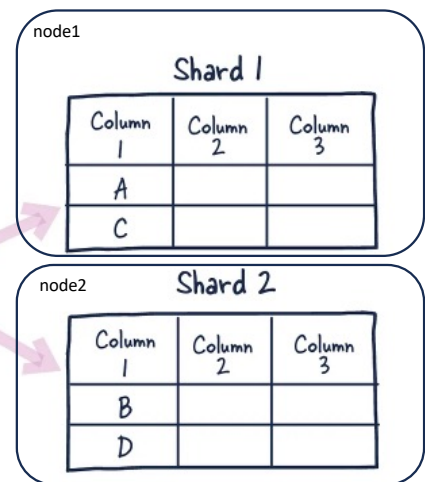
Shard Key

Column 1	Column 2	Column 3
A		
B		
C		
D		

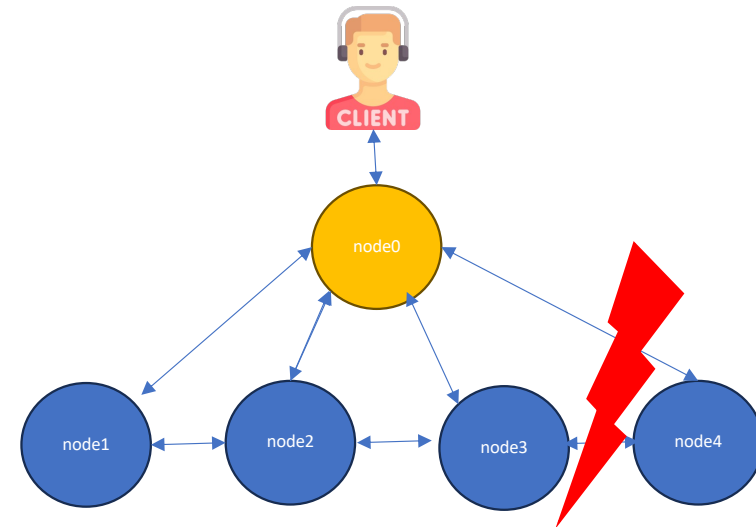
Hash
function

Column 1	Hash values
A	1
B	2
C	1
D	2

Key Based Sharding



Théorème de CAP



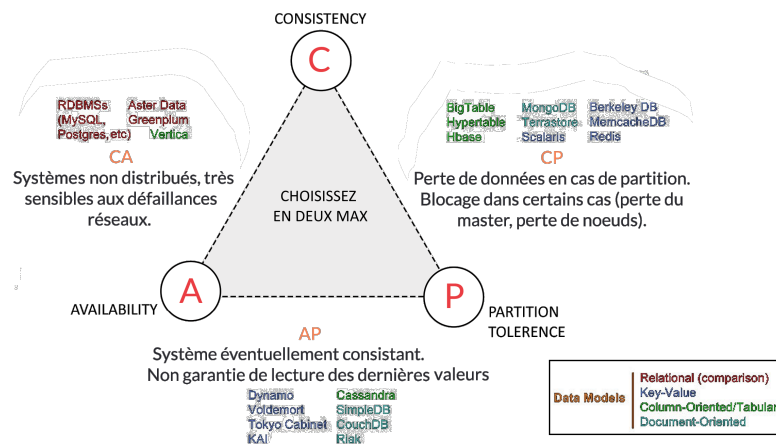
Imaginons un jeu de données réparti sur plusieurs nœuds d'un système distribué.

En cas de panne ou de coupure réseau entre ces nœuds (ce qu'on appelle une partition), le système doit faire un choix :

- ☐ Le système continue de répondre (disponibilité prioritaire) mais certaines réponses peuvent être obsolètes ou incohérentes, car tous les nœuds ne sont pas synchronisés. On parle alors d'un système **AP** (Availability + Partition tolerance).
- ☐ Le système attend de garantir la cohérence entre les nœuds, il bloque la réponse tant que les données ne sont pas synchroniser entre tous les nœuds. C'est le comportement d'un système **CP** (Consistency + Partition tolerance)

Théorème de CAP

- ❑ Le scientifique Eric Brewer constate que tous les systèmes distribués de stockage de données ne peuvent garantir que 2 des 3 propriétés suivantes :
- **Consistency** : Une lecture retourne toujours la dernière valeur écrite, quelle que soit la machine interrogée.
 - **Availability** : Le système répond à chaque requête, même en cas de panne partielle.
 - **Partition tolerance** : Le système continue de fonctionner même si une partie du réseau est coupée.



Les systèmes boursiers ?

CA

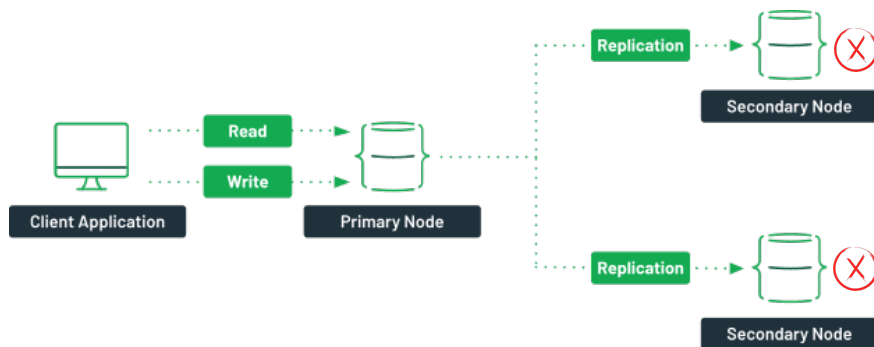
Des réseaux de guichets automatiques ?

CP

La gestion de traces applicatives ?

PA

La gestion de la cohérence : Eventual consistency



Facteur de réplication FR : 3

Validation en écriture W : 1

Le système confirme l'écriture dès que l'écriture sur le primaire est validée et la réplication sur les 2 nœuds restants se fait de manière asynchrone.

Facteur de réplication FR : 3

Validation en lecture R : 1

Le système renvoie le résultat à partir de n'importe quel nœud.
Résultat éventuellement incohérent.

Facteur de réplication FR : 3

Validation en lecture R : 2

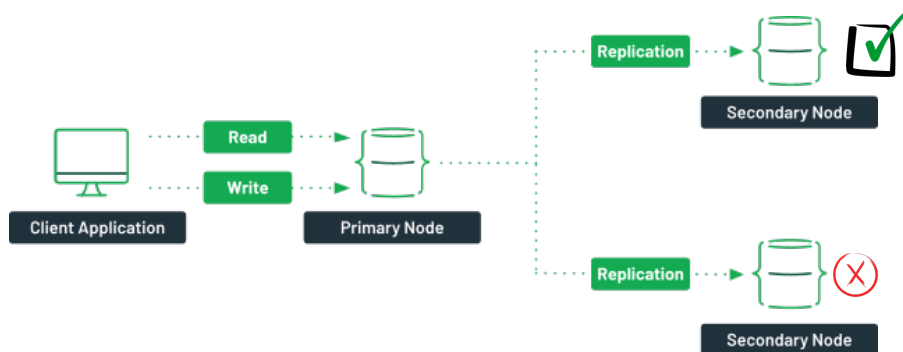
Le système renvoie le résultat en se mettant d'accord sur le résultat avec un second nœud.
Résultat éventuellement incohérent.

Facteur de réplication FR : 3

Validation en lecture R : 3

Le système renvoie le résultat en se mettant d'accord entre les 3 nœuds.
Résultat cohérent

La gestion de la cohérence : Eventual consistency



Facteur de réplication FR : 3

Validation en écriture W : 2

Le système confirme l'écriture dès que l'écriture sur le primaire est validée et que la réplication est validée sur au moins 1 réplica.

La cohérence des données lues est établie selon le niveau de cohérence demandé en écriture :

$W + R > FR$

Facteur de réplication FR : 3

Validation en lecture R : 1

Le système renvoie le résultat à partir de n'importe quel nœud.
Résultat éventuellement incohérent.

Facteur de réplication FR : 3

Validation en lecture R : 2

Le système renvoie le résultat en se mettant d'accord sur le résultat avec un second nœud.
Résultat cohérent.

Facteur de réplication FR : 3

Validation en lecture R : 3

Le système renvoie le résultat en se mettant d'accord entre les 3 nœuds.
Résultat cohérent

SGBD Relationnel où NoSQL ?

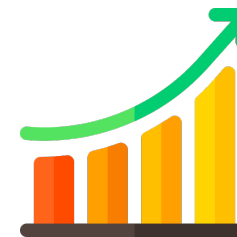
Relationnel



Cohérence



OU



Extensibilité

NOSQL

SGBD Relationnel où NoSQL ?

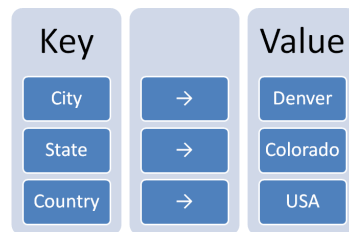
- ☐ **ACID vs BASE : faut-il renoncer à la rigueur dans les systèmes distribués ?**
- ☐ Dans les bases de données **relationnelles classiques**, les **règles ACID** garantissent un comportement fiable et cohérent des transactions.
- ☐ Sans un **environnement distribué à grande échelle**, respecter toutes les garanties ACID devient **très complexe** notamment à cause de la **latence réseau**, des **pannes** et de la **répartition des données**.
- ☐ Cela a conduit à la popularisation du **modèle BASE**, adopté par de nombreuses bases **NoSQL**.

SGBD Relationnel où NoSQL ?

- ☐ **B - Basically Available** : Le système reste **généralement disponible**, même en cas de panne.
- ☐ **S - Soft State** : L'état du système peut évoluer au fil du temps, il est possible que plusieurs nœuds aient temporairement des **états différents**.
- ☐ **Eventually consistent** : À terme, tous les nœuds du système finiront par converger vers un état cohérent

Le NoSQL

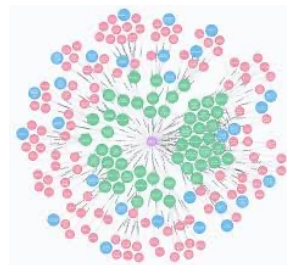
Orienté Clé-Valeur



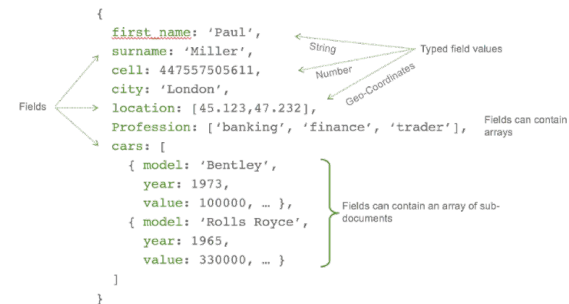
Orienté Colonne

train6101	PARIS 06:07	VALENCE 08:19	AVIGNON 09:07	AIX 09:30	MARSEILLE 09:41
train2917			AVIGNON 11:30		11:59 MARSEILLE

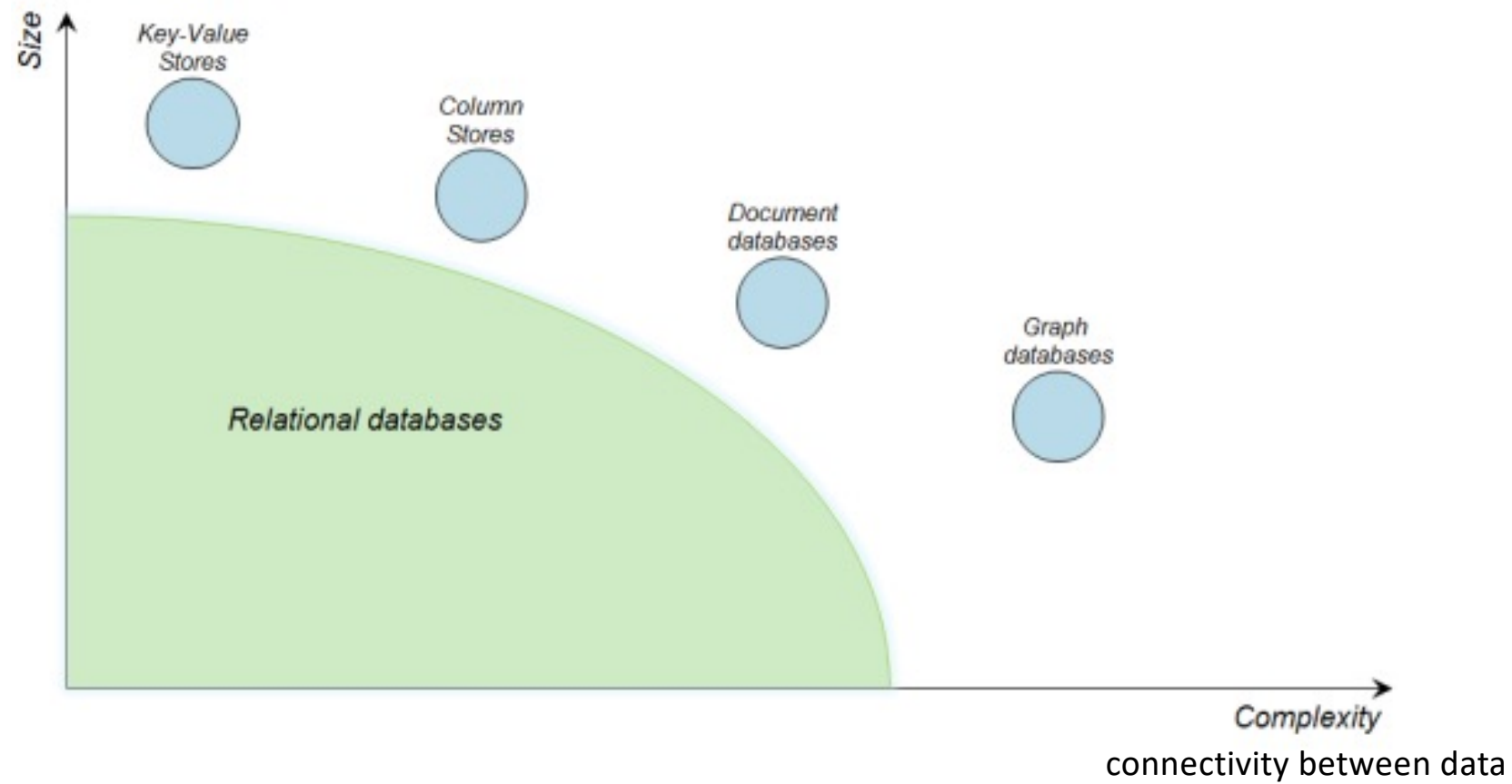
Orienté Graphe



Orienté Document



SGBD Relationnel où NoSQL ?

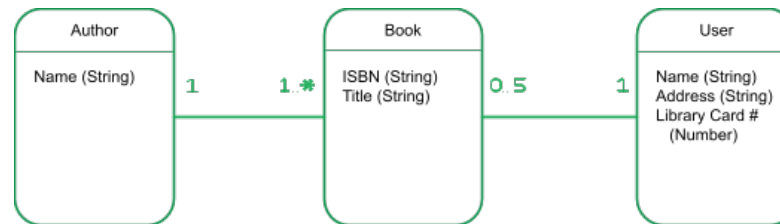


Le modèle relationnel

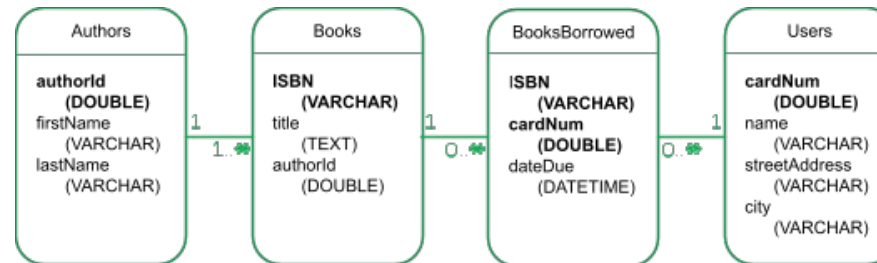
Modèle conceptuel



Modèle logique

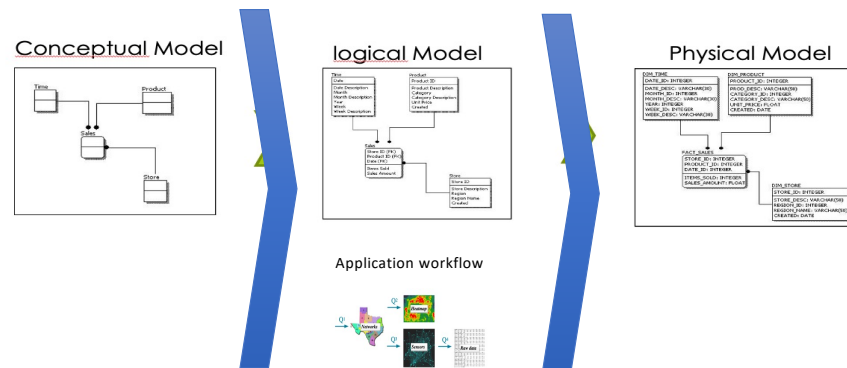


Modèle Physique



@mongoDB

Le modèle NoSQL



❑ Modélisation NoSQL : une approche orientée performance

❑ Query-Driven Design

- Le modèle de données est conçu à **partir des requêtes** fréquentes de l'application, afin de **minimiser les lectures coûteuses** et éviter les jointures.

❑ Dénormalisation assumée

- Pour améliorer les performances, on **duplique** ou **imbrique** les données.
- Moins de relations, plus d'efficacité en lecture.

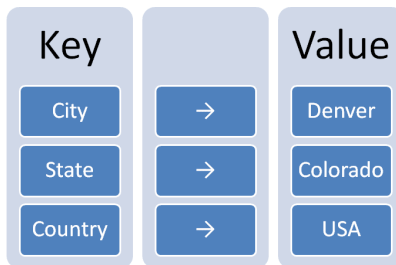
❑ Flexibilité du schéma

- Les bases NoSQL permettent des **modifications structurelles sans migration**, ce qui facilite les **évolutions rapides** du modèle dans des contextes agiles.

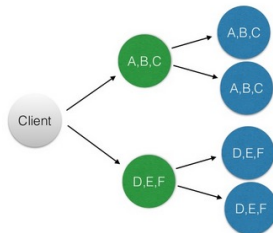
❑ Modèle utile mais pas parfait

- On ne cherche pas un modèle parfait, mais un équilibre pratique entre performance, scalabilité et exigences métiers.

Le NoSQL : la famille orientée clé/valeur



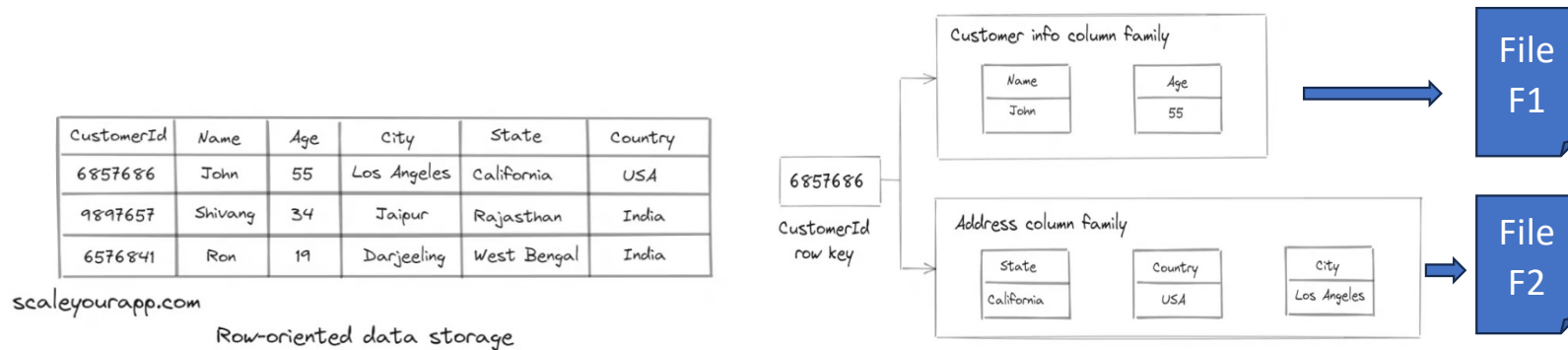
Redis Cluster
Sharding and replication (asynchronous).



- ☐ Données stockées sous forme de paires clé → valeur
- ☐ Accès ultra-rapide par la clé uniquement
- ☐ Aucun schéma ou structure imposée pour la valeur (souvent un "blob")
- ☐ Possibilité de simuler des objets (user:123:name → "Alice")
- ☐ En mémoire (ex: Redis) ou sur disque (ex: Riak, DynamoDB)
- ☐ Scalabilité facile via sharding
- ☐ Cas d'usage typiques :
 - Cache d'objets, cookies

Le NoSQL : la famille orientée colonne

- ❑ Le terme “base orientée colonne” est utilisé de manière ambiguë selon qu’on parle de modèle logique (comme dans HBase) ou de stockage physique (comme dans ClickHouse, etc.).
- ❑ Au niveau logique, les bases reposent sur une fragmentation verticale des données au lieu de stocker les lignes ensemble, on stocke les colonnes séparément.
- ❑ Une famille de colonnes regroupe un ensemble de colonnes souvent accédées ensemble.
 - Chaque famille est stockée physiquement de manière indépendante, afin d’améliorer la performance.



Le NoSQL : la famille orientée colonne

Row-Oriented vs Column-Oriented



Row-oriented: rows stored sequentially in a file

Key	Fname	Lname	State	Zip	Phone	Age	Sales
1	Bugs	Bunny	NY	11217	(123) 938-3235	34	100
2	Yosemite	Sam	CA	95389	(234) 375-6572	52	500
3	Daffy	Duck	NY	10013	(345) 227-1810	35	200
4	Elmer	Fudd	CA	04578	(456) 882-7323	43	10
5	Witch	Hazel	CA	01970	(567) 744-0991	57	250

Column-oriented: each column is stored in a separate file
Each column for a given row is at the same offset.

Key	Fname	Lname	State	Zip	Phone	Age	Sales
1	Bugs	Bunny	NY	11217	(123) 938-3235	34	100
2	Yosemite	Sam	CA	95389	(234) 375-6572	52	500
3	Daffy	Duck	NY	10013	(345) 227-1810	35	200
4	Elmer	Fudd	CA	04578	(456) 882-7323	43	10
5	Witch	Hazel	CA	01970	(567) 744-0991	57	250

- ❑ Chaque colonne est stockée indépendamment sur disque.
- ❑ Grâce à la similarité des valeurs dans une colonne, les taux de compression sont optimisés.
- ❑ Un usage optimisé pour les applications **analytique** et lecture de colonnes sur de grands volumes (ex Click House, **BigQuery**)

Le NoSQL : la famille orientée Graphe

❑ Principe

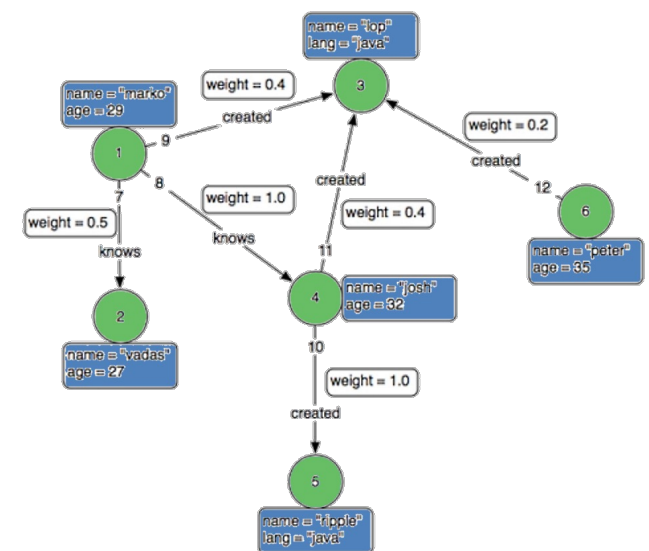
- Fondées sur la théorie des graphes
- Les données sont représentées par des nœuds (entités) et des relations (arcs orientés)
- Relations et nœuds peuvent porter des propriétés

❑ Idéal pour les données très connectées :

- Réseaux sociaux
- Recommandation

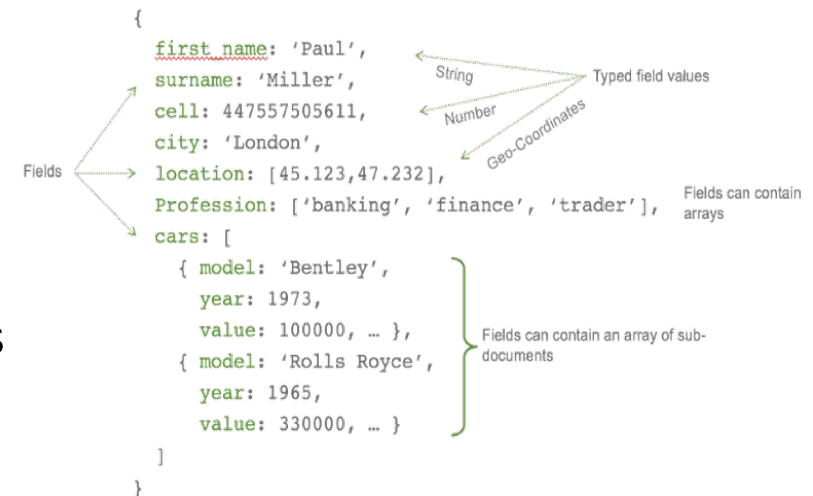
❑ Limites

- Scalabilité horizontale complexe

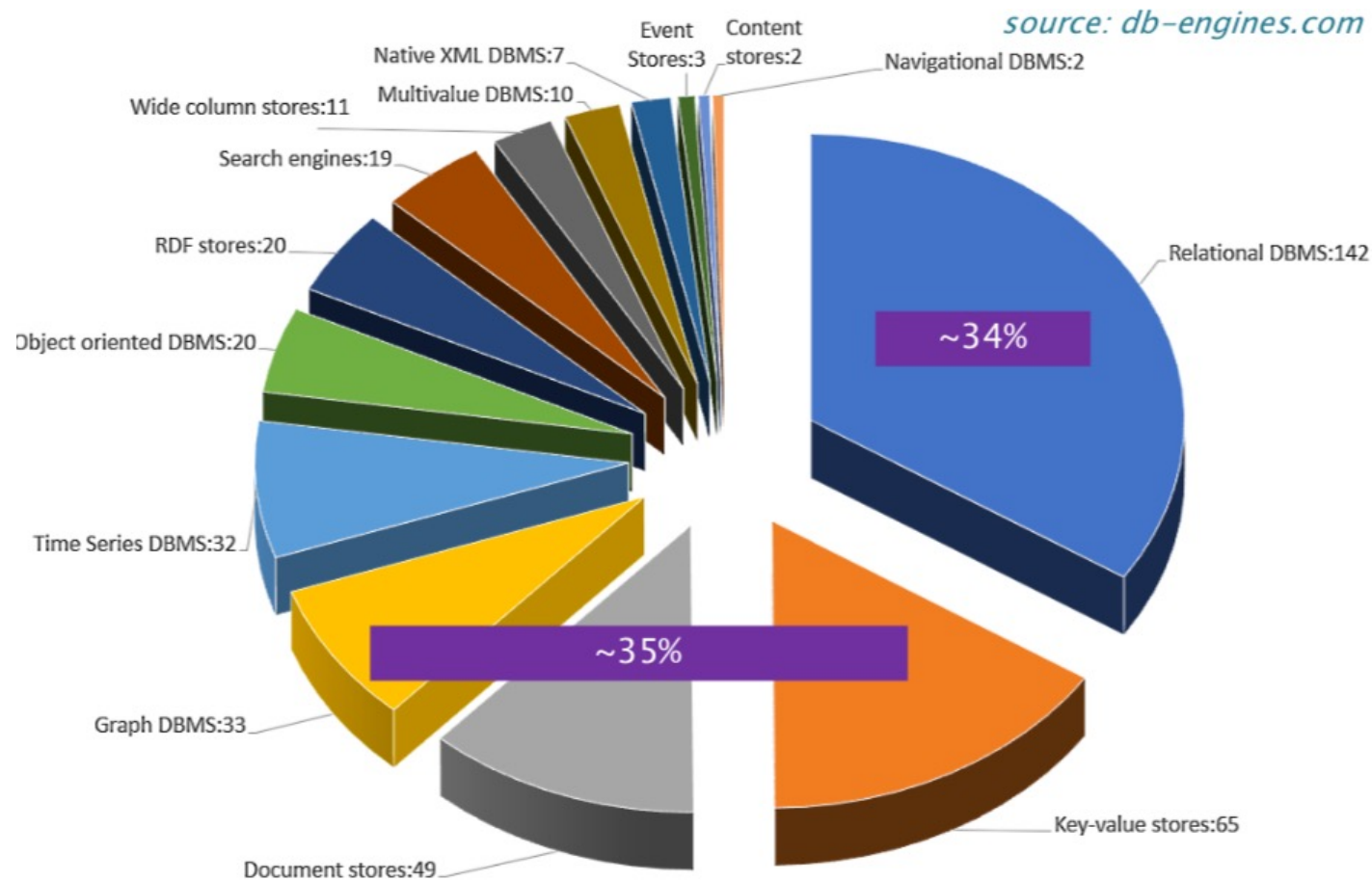


Le NoSQL : la famille orientée Document

- ❑ Les données sont stockées sous forme de documents structurés (généralement en JSON, BSON ou XML).
- ❑ Un document est un ensemble de paires clé/valeur, où les valeurs peuvent être :
 - des types simples (chaîne, nombre...)
 - ou des objets imbriqués (documents ou tableaux)
- ❑ On est amené à dupliquer volontairement des données (dénormalisation) pour optimiser la lecture
- ❑ Ce modèle favorise les lectures rapides.
- ❑ Ex : MongoDB, CouchDB, ArangoDB, DocumentDB

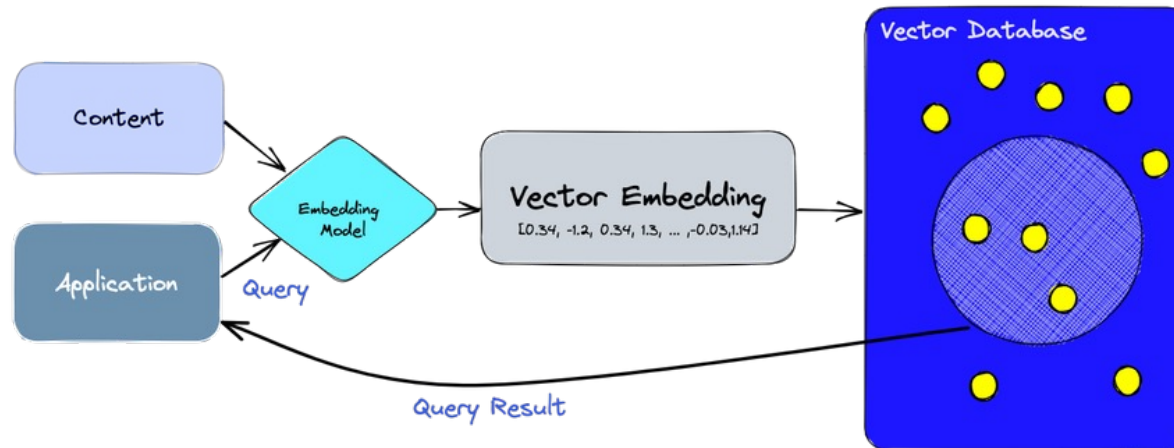


Le NoSQL aujourd'hui (415 DB)

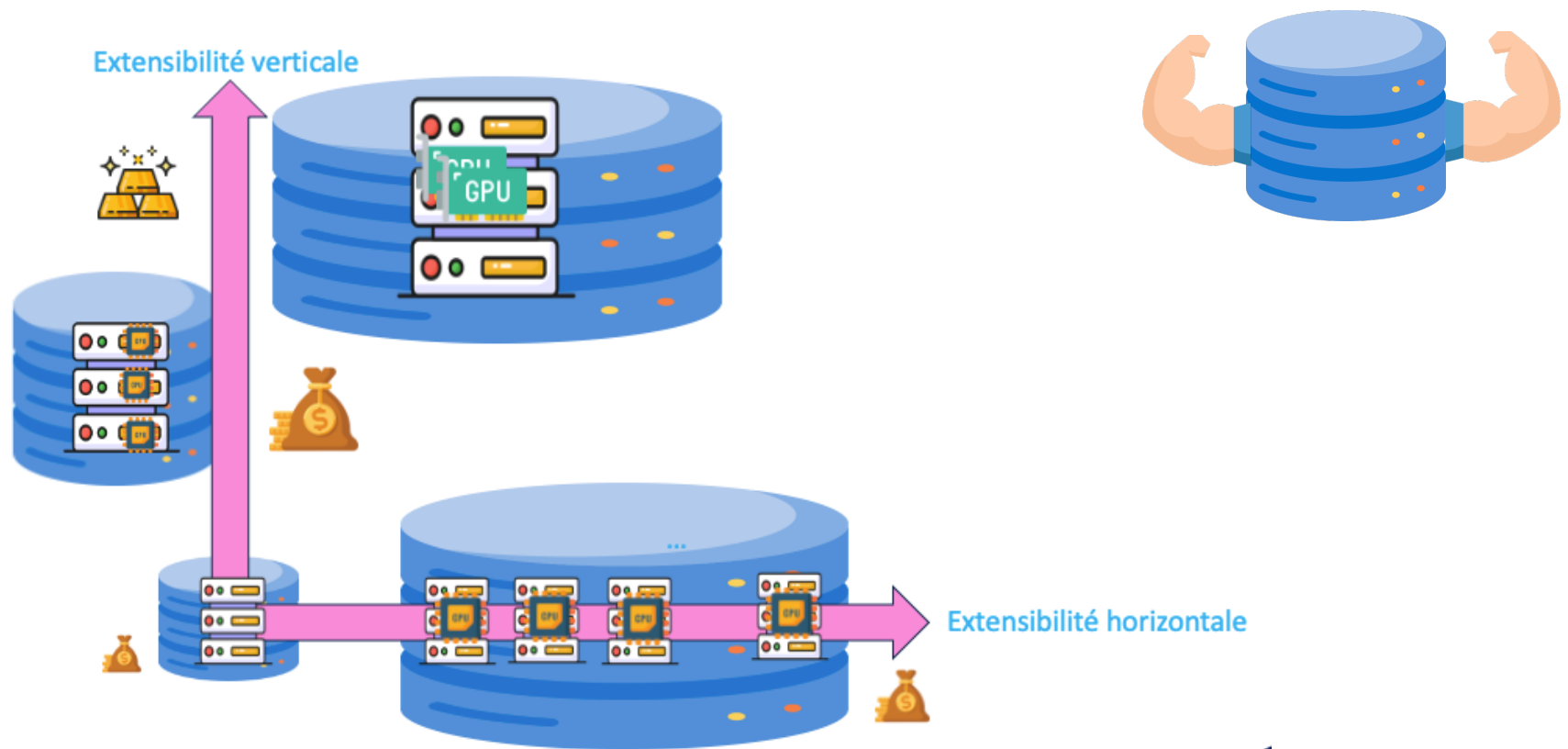


Et demain !

- ❑ Une base de données vectorielle est un type de base de données spécialement conçue pour stocker et manipuler des données vectorielles de manière efficace.
- ❑ La recherche vectorielle est une méthode pour représenter et rechercher vos objets (documents, chansons, images, ...) dans un espace géométrique (généralement de haute dimension).

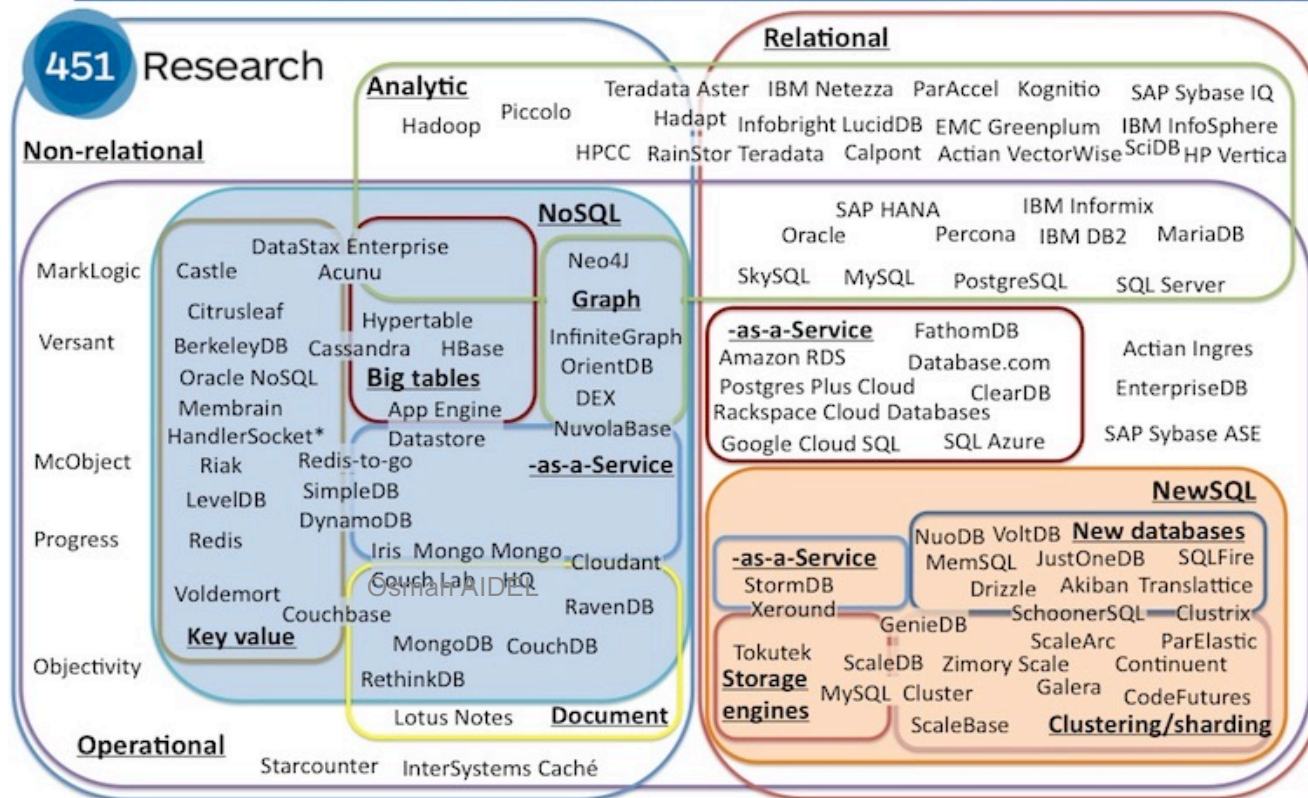


Et après demain !

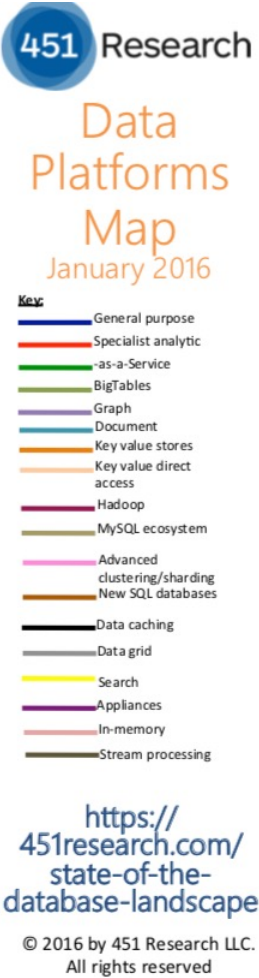


Vision des bases de données en 2012

The evolving database landscape



Vision des plateformes de données en 2016



Le futur ?

BIG DATA STORYMAP

EMC²

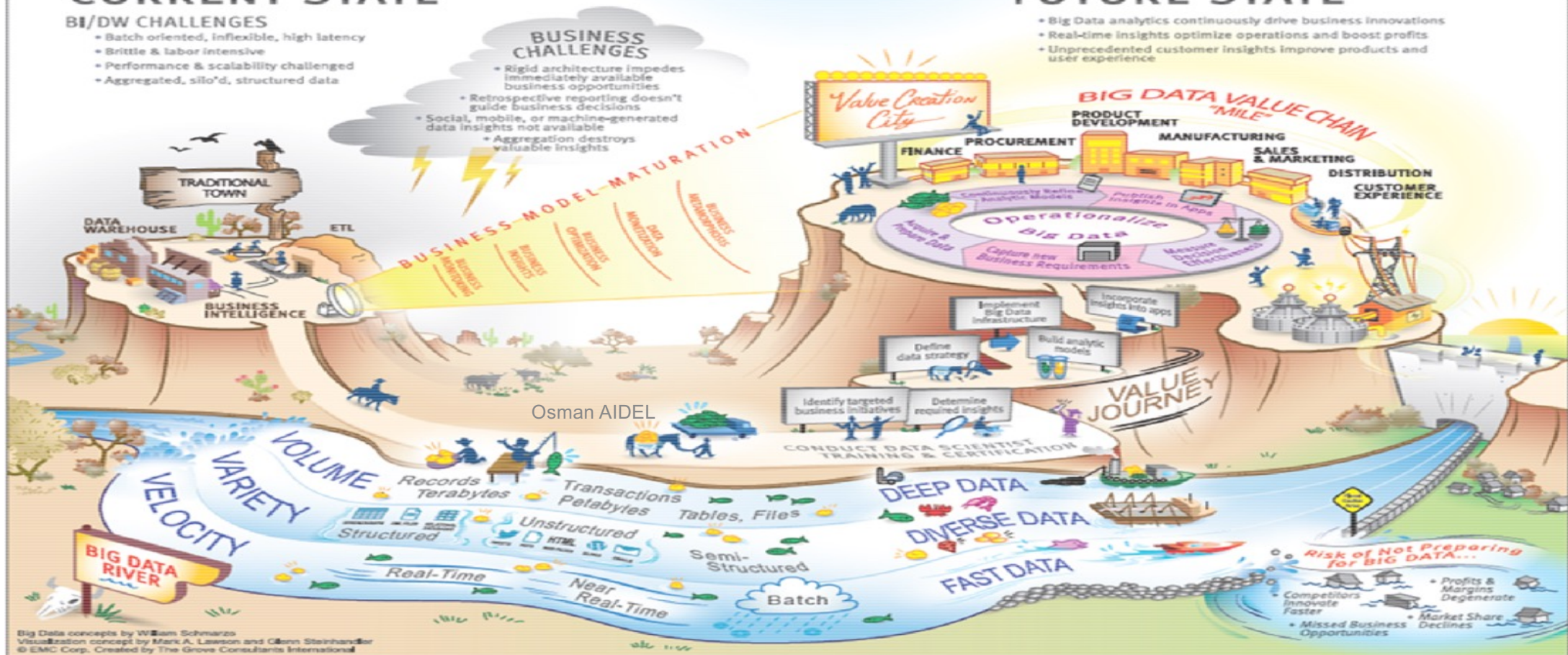
CURRENT STATE

BI/DW CHALLENGES

- Batch oriented, inflexible, high latency
- Brittle & labor intensive
- Performance & scalability challenged
- Aggregated, silo'd, structured data

BUSINESS CHALLENGES

- Rigid architecture impedes immediately available business opportunities
- Retrospective reporting doesn't guide business decisions
- Social, mobile, or machine-generated data insights not available
- Aggregation destroys valuable insights



Osman AIDEL